



HiLCoE

School of Computer Science and Technology

FLEET MANAGEMENT SYSTEM

SENIOR PROJECT OUTLINE DOCUMENT

Prepared by:

MELEK BEKELE

MINTESNOT ABEBE

YONAS BERHANU

ZEKARIAS TESFAYE

AUGUST 2026

FLEET MANAGEMENT SYSTEM

Prepared by

MELEK BEKELE

MINTESNOT ABEBE

YONAS BERHANU

ZEKARIAS TEFAYE

**A SENIOR PROJECT DOCUMENT SUBMITTED TO THE UNDERGRADUATE
PROGRAMME OFFICE IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE IN
SOFTWARE ENGINEERING**

ADVISOR: MESFIN KIFLE (PHD)

AUGUST 2026

HiLCoE

School of Computer Science and Technology

FLEET MANAGEMENT SYSTEM

Prepared by

MELEK BEKELE

MINTESNOT ABEBE

YONAS BERHANU

ZEKARIAS TESFAYE

Approved by:

Advisor:

Signature:

Examiner:

Signature:

Table of Contents

1. Introduction/Background	1
Past Fleet Management Systems	1
• Manual Records and Paper-Based Management	1
• Spreadsheets and Early Computerization	2
• Radio Dispatching and Early Communication	2
• Early GPS and Telematics	2
• Basic Vehicle Diagnostics	3
• Limitations of Past Systems	3
Present-Day Fleet Management Systems	3
• Integrated Telematics Architecture	3
• Real-Time GPS Tracking and Dynamic Routing	4
• Driver Behavior Analytics	4
• Predictive Maintenance	4
• Fuel and Energy Management	5
• IoT, Vehicle Diagnostics, and Cloud Platforms	5
• Automation and Reporting	5
• Existing Commercial Platforms	6
Future and Emerging Fleet Management Systems	7
• AI-Driven Predictive Maintenance	7
• Digital Twins	7
• Edge Computing	7
• Autonomous Fleets and Sensor Fusion	8
• AI-Based Fleet Optimization and Autonomous Decision-Making	8
3. General and Specific Objectives	11
3.1 General Objective	11
3.2 Specific Objectives	12
4. Scope & Limitations	13
4.1 Scope	14
Out of Scope	16
4.2 Limitations	16
5. Methodology / Approach	17
5.1 Development Methodology	17
5.2 Requirements Gathering and Analysis	18
5.3 System Design	18
5.4 System Implementation	19
5.5 Environment Configuration	19

5.6 Testing and Quality Assurance	20
5.7 Deployment and Documentation	20
6. Significance & Beneficiaries	21
6.1 Significance of the Project	21
6.2 Beneficiaries of the Project	22
7. Task Breakdown & Feasibility	23
7.1 Task Breakdown	24
7.2 Feasibility Analysis	38
8. Project Schedule / Timeline	43
8.1 PERT Three-Point Duration Estimates	44
8.2 Phase-Level Gantt Schedule	45
8.3 MVP Implementation Schedule (Phase 3A Detail)	46
8.4 Phase-2 Implementation Schedule (Phase 3B Detail — Contingent)	47
8.5 Out-of-Scope Implementation Schedule (Phase 3C Detail)	47
8.6 Critical Path & Float Analysis	48
8.7 Key Milestones	49
9. References	50

1. Introduction/Background

Fleet management has evolved from manual administrative processes into an increasingly connected, data-driven, and intelligent operational discipline. Traditionally, fleet operators relied on paper records, driver reports, telephone or radio communication, spreadsheets, and scheduled maintenance. The introduction of computerized information systems, GPS, mobile communication, and vehicle telematics progressively improved the ability to monitor and coordinate vehicles. More recently, cloud computing, Internet of Things (IoT) technologies, vehicle diagnostics, predictive analytics, and artificial intelligence (AI) have transformed fleet management into a real-time decision-support environment.

The evolution of fleet management can broadly be divided into three stages. Past systems were primarily manual, centralized, and reactive; present-day systems are connected, automated, and increasingly predictive; while future systems are expected to incorporate AI, digital twins, edge computing, autonomous vehicles, sensor fusion, and electric-vehicle (EV) optimization. These stages are not strictly chronological because organizations continue to operate combinations of legacy and modern technologies. Nevertheless, they provide a useful framework for understanding how fleet-management capabilities have developed.

The central technological transition is from recording what happened, to monitoring what is happening, and ultimately toward predicting what will happen and automatically determining what should happen next.

Past Fleet Management Systems

- **Manual Records and Paper-Based Management**

Early fleet management was predominantly administrative. Fleet managers maintained paper files containing vehicle registration information, maintenance records, inspection reports, fuel receipts, mileage, driver assignments, and accident documentation. Vehicle locations were generally determined through driver reports, scheduled check-ins, physical observation, or communication with dispatchers.

The resulting workflow was essentially: Vehicle → Driver → Dispatcher → Manual record → Supervisor → Decision. Although simple, this approach created considerable information delays. A vehicle problem might only become known after the driver contacted the dispatcher, while maintenance decisions depended heavily on mileage records, inspections, and driver observations. Consequently, management was predominantly reactive rather than predictive.

The broader development of computerized fleet management followed the emergence of centralized computing. Early systems used mainframe computers and text-based terminals to replace some paper-based administrative processes. Such systems improved data storage and reporting, but processing remained centralized and access was generally restricted to specialized terminals (Automotive Fleet, n.d.).

- **Spreadsheets and Early Computerization**

The adoption of spreadsheets represented an important step toward digital fleet administration. Fleet managers could calculate mileage, fuel consumption, maintenance costs, and service intervals more efficiently than with handwritten records. However, spreadsheets primarily digitized existing processes rather than fully automating them.

- **Radio Dispatching and Early Communication**

Radio communication substantially improved fleet coordination by allowing drivers and dispatchers to communicate while vehicles were in operation. Dispatchers could receive information about delays, breakdowns, route changes, or vehicle locations without waiting for the vehicle to return to a depot. However, radio communication improved communication rather than automated information generation. A dispatcher still depended on the driver to provide accurate information, and the information normally had to be manually entered into another system.

- **Early GPS and Telematics**

The introduction of GPS represented a major transition because vehicle location could increasingly be determined automatically. Fleet managers could move from relying on driver-reported positions to receiving machine-generated geographic coordinates. Early GPS

applications primarily focused on vehicle tracking and navigation, while later telematics systems combined positioning with wireless communication and vehicle information. Goel (2008) demonstrated how fleet telematics could integrate vehicle-positioning technologies, wireless communication, management information systems, and dynamic vehicle-routing algorithms. This enabled real-time monitoring and decision support when transportation conditions changed.

- **Basic Vehicle Diagnostics**

The development of electronic control units and standardized vehicle communication protocols further expanded fleet monitoring. Instead of relying exclusively on physical inspections, vehicles could generate information electronically. Heavy-duty vehicle standards such as SAE J1939 provide communication and diagnostic mechanisms between electronic control modules. These technologies established an important technical foundation for modern vehicle-health monitoring. However, early diagnostic systems generally provided information rather than intelligence: a diagnostic trouble code could indicate that a problem existed, but determining its cause, severity, and appropriate maintenance action still depended heavily on technicians.

- **Limitations of Past Systems**

The principal limitations of historical fleet systems were limited automation, fragmented information, delayed visibility, reactive maintenance, limited analytics, and weak integration. Data collection and reporting required substantial human involvement; information was distributed across paper files, spreadsheets, radio reports, and separate databases; and maintenance was commonly based on fixed intervals or failures rather than predicted component condition. These limitations established the technological requirements that modern fleet-management platforms attempt to address.

Present-Day Fleet Management Systems

- **Integrated Telematics Architecture**

Modern fleet-management systems combine multiple technologies into an integrated platform. A typical architecture can be represented as: Vehicle sensors/ECUs → Telematics gateway → Wireless network → Cloud platform → Database/analytics → Dashboard/API → Automated

action. This architecture combines GPS, vehicle diagnostics, IoT sensors, mobile communication, cloud computing, data analytics, and workflow automation. Goel (2008) identified the importance of integrating telematics with management information systems and dynamic decision-support capabilities. Modern systems extend this concept through cloud infrastructure and large-scale data analytics.

- **Real-Time GPS Tracking and Dynamic Routing**

Real-time GPS is now one of the fundamental capabilities of fleet-management systems. Modern platforms can provide current vehicle position, speed and direction, route progress, geofence events, route deviations, estimated arrival times, historical routes, and stop and idle information. The significance of GPS therefore extends beyond visualization. Vehicle location can be incorporated into dispatching and dynamic routing decisions. Pillac et al. (2013) demonstrate that dynamic vehicle-routing research explicitly considers information that becomes available during route execution. Consequently, modern fleet management can respond to changing conditions rather than simply following a static plan.

- **Driver Behavior Analytics**

Modern telematics systems increasingly analyze how vehicles are driven, rather than merely where they are located. Typical variables include speed, acceleration, braking, idling, cornering, and distance. Walnum and Simonsen (2015) analyzed heavy-duty truck fleet-management data and found that driving behavior influences fuel consumption, although vehicle and infrastructure characteristics can also have substantial effects. A systematic review by Boylan et al. (2024) found that machine learning is increasingly used to analyze telematics data and that speed, braking, acceleration, and distance are among the most frequently used variables. This allows fleet systems to produce driver-performance indicators and safety scores, supporting targeted coaching and risk management.

- **Predictive Maintenance**

Predictive maintenance is one of the most significant developments in modern fleet management. Traditional maintenance followed fixed schedules: mileage/time threshold → inspection →

service. Predictive maintenance instead uses vehicle data to identify anomalies and estimate potential failures: continuous monitoring → anomaly detection → failure prediction → maintenance planning. Killeen et al. (2019) developed an IoT-based predictive-maintenance architecture for public-transport buses using J1939 sensor data and semi-supervised machine learning. Their research demonstrates how fleet-level vehicle-health data can be used to identify vehicles that deviate from normal operating patterns. This approach can reduce dependence on fixed maintenance intervals and potentially improve vehicle availability, although high-quality failure data remain scarce.

- **Fuel and Energy Management**

Modern fleet systems can combine fuel transactions with vehicle and telematics information, including fuel purchases, fuel level, mileage, engine hours, idling, vehicle load, route, speed, and driving behavior. Walnum and Simonsen (2015) demonstrate the usefulness of fleet-management data for analyzing fuel consumption and identifying the influence of vehicle, infrastructure, and driver-related variables. This enables managers to identify inefficient vehicles, excessive idling, inefficient driving practices, and routes with high energy consumption. The same concept is increasingly important for EV fleets, where fuel management becomes energy management.

- **IoT, Vehicle Diagnostics, and Cloud Platforms**

IoT enables fleet-management systems to collect information from a wide range of sensors, including vehicle systems, tires, cargo compartments, refrigeration units, batteries, and environmental sensors. Cloud computing provides centralized storage and processing, allowing authorized users to access fleet data through web dashboards and mobile applications. Modern APIs also allow fleet systems to exchange data with ERP systems, accounting systems, maintenance applications, fuel-card systems, warehouse systems, customer applications, and external analytics platforms. Consequently, the modern FMS is increasingly an ecosystem rather than an isolated application.

- **Automation and Reporting**

Modern systems can automatically generate alerts and workflows when predefined conditions occur. For example: diagnostic fault detected → alert generated → maintenance task created → technician notified → vehicle scheduled → repair recorded. Similarly, a geofence violation can be detected automatically, followed by notification and incident recording. Modern dashboards also transform fleet information into key performance indicators such as vehicle utilization, fuel efficiency, maintenance cost, downtime, driver performance, route efficiency, and fleet availability.

- **Existing Commercial Platforms**

1. **Webfleet** provides vehicle tracking, professional navigation, route optimization, video telematics, vehicle diagnostics, driver-behavior analysis, fuel-saving tools, and AI-supported reporting. Its capabilities illustrate the integration of operational tracking with driver performance, vehicle health, and route optimization (Webfleet, 2026).
2. **Motive** integrates GPS fleet telematics with driver-safety monitoring, vehicle diagnostics, fuel and electric-vehicle management, compliance, and workflow automation. Its platform emphasizes the consolidation of telematics, video, vehicle, and operational data to reduce manual work and support fleet-level decision-making (Motive, 2026).
3. **Verizon Connect** provides near-real-time GPS tracking together with fuel and cost management, driver-safety monitoring, asset maintenance, alerts, and integrations for dispatching, scheduling, and work orders. Its platform demonstrates how location, driver, vehicle-health, and fuel information can be presented through a centralized operational interface (Verizon Connect, 2026).
4. **Samsara** provides an integrated connected-operations platform combining real-time GPS tracking, vehicle diagnostics, routing and dispatch, maintenance, fuel and energy management, driver safety, mobile workflows, reporting, APIs, and integrations. Its platform also includes tools for electric vehicles and equipment monitoring, illustrating the transition from basic GPS tracking toward broader fleet and operations management (Samsara, 2026).
5. **Geotab** provides a connected-fleet platform that can ingest data from its own telematics devices, OEM-connected vehicles, third-party devices, sensors, and other data streams.

Its platform emphasizes real-time vehicle and asset data, analytics, artificial intelligence, machine learning, and an open ecosystem for integration (Geotab, 2026).

Future and Emerging Fleet Management Systems

- **AI-Driven Predictive Maintenance**

The next generation of predictive maintenance is expected to combine multiple sources of information rather than relying only on diagnostic codes. Potential inputs include vehicle sensor data, maintenance history, driving behavior, weather, road conditions, vehicle age, payload, traffic, and operating environment. Machine-learning models can then estimate the probability of component failure or remaining useful life. The resulting process could be: data collection → feature extraction → anomaly detection → failure prediction → risk assessment → maintenance optimization. This represents a shift from simply identifying faults toward predicting when and under what operating conditions a failure is likely to occur.

- **Digital Twins**

A digital twin is a virtual representation of a physical system that can be continuously updated using operational data. In fleet management, a digital twin could represent vehicle condition, component health, location, route, energy consumption, maintenance history, and operational availability. Digital-twin research for connected vehicles demonstrates potential applications in monitoring, simulation, resource management, and optimization. At the fleet level, digital twins could represent vehicles, drivers, routes, workshops, depots, and charging infrastructure, allowing managers to conduct what-if simulations before implementing operational changes.

- **Edge Computing**

Cloud computing is appropriate for large-scale historical analytics, but some fleet applications require very low response times. Edge computing addresses this problem by processing selected data closer to the vehicle. A future architecture could therefore be: vehicle sensors → edge AI → immediate local decision, while edge data are simultaneously sent to the cloud for fleet-wide analytics. This can reduce latency and communication requirements for time-sensitive applications and is particularly relevant when fleets incorporate autonomous-driving functions, ADAS, or high-frequency sensor data.

- **Autonomous Fleets and Sensor Fusion**

Future fleet-management systems will increasingly have to manage vehicles with sophisticated perception and automated-driving capabilities. Autonomous vehicles commonly use combinations of cameras, radar, LiDAR, GPS, inertial sensors, and other environmental sensors. Yeong et al. (2021) identify sensor calibration and sensor fusion as fundamental requirements for reliable autonomous-vehicle perception. Fayyad et al. (2020) review deep-learning approaches to sensor fusion for autonomous-vehicle perception and localization, emphasizing the advantages of combining complementary sensing modalities. For fleet management, this introduces a new maintenance dimension: managers may need to monitor cameras, LiDAR, radar, computing units, software versions, calibration status, and cybersecurity in addition to traditional mechanical components.

- **AI-Based Fleet Optimization and Autonomous Decision-Making**

The future fleet manager may increasingly supervise algorithmic decisions rather than manually assigning every vehicle. Research on connected and autonomous vehicles is already examining optimization of routing, traffic management, platooning, trajectory planning, and energy efficiency. Nisyrios and Gkiotsalitis (2025) found rapid growth in optimization-based connected and autonomous fleet management research, with increasing attention to real-time data, artificial intelligence, reinforcement learning, and mixed-traffic environments. A future intelligent fleet platform could therefore operate a continuous loop: Sense → Predict → Optimize → Act → Monitor → Re-optimize.

2. Statement of the Problem

Transport operators in Ethiopia currently manage their fleet operations through largely manual, disconnected, or partially digitized processes that fail to provide a comprehensive, real-time view of vehicle, driver, and operational performance. An examination of current practice across the functional areas relevant to this project reveals a series of interrelated deficiencies, each of which contributes to elevated costs, safety risks, and reduced productivity across the fleet lifecycle.

Fragmented driver and vehicle records. Vehicle profiles (registration details, specifications, condition history) and driver profiles (licensing, contact, and performance information) are typically maintained in separate, non-standardized formats—or not maintained in structured form at all. This fragmentation complicates vehicle–driver assignment decisions, as managers lack a consolidated reference for matching driver qualifications to vehicle requirements. Attendance is rarely tracked systematically, allowing unexcused absences to disrupt trip schedules without early detection. Driver performance is seldom scored against objective criteria, so risk-prone or underperforming drivers are not identified until an incident occurs. Daily Vehicle Inspection Reports (DVIRs), where conducted, are recorded on paper or omitted altogether, meaning early indicators of mechanical defects frequently go unreported until failure. Accident reporting is inconsistent in format and timing, and expiry dates for licenses, insurance, and roadworthiness documentation are tracked manually if at all—exposing operators to compliance violations and the risk of operating vehicles or dispatching drivers with lapsed credentials.

Inefficient trip scheduling and limited route visibility. Trip assignments are commonly coordinated through informal channels such as phone calls or handwritten dispatch sheets, increasing the likelihood of scheduling conflicts and missed trips. Without route optimization, drivers frequently select routes based on familiarity rather than distance, traffic, or fuel efficiency, resulting in avoidable time and cost overruns. Mileage is largely self-reported, making it difficult to reconcile against fuel consumption or maintenance intervals. The absence of ETA monitoring leaves dispatchers and clients without visibility into a trip's progress until a delay has already occurred, and delay reporting—when it happens—rarely captures root causes in a form

suitable for later analysis. Consequently, operators lack the trip-level analytics needed to identify consistently inefficient routes or underperforming drivers.

Untracked fuel consumption and cost leakage. Fuel purchases are commonly recorded through receipts or logbooks that are prone to loss, delay, or manipulation. Because consumption is not systematically reconciled against distance traveled, operators lack a reliable cost-per-kilometer figure for individual vehicles, obscuring one of the largest recurring operating expenses in fleet management. Efficiency trends—such as gradual declines linked to engine wear or driving behavior—are not visible without consistent tracking, and no mechanism exists to flag sudden or abnormal consumption spikes. As a result, fuel theft, siphoning, or leakage frequently goes undetected for extended periods, representing a direct and largely invisible financial loss.

Absence of proactive alerting mechanisms. Critical operational events—including approaching maintenance due dates, document and license expiries, speeding, excessive idling, and unauthorized vehicle usage—are presently detected only after the fact, if at all, since no centralized mechanism exists to monitor and flag these conditions in real time. This reactive posture means that preventable incidents, from compliance violations to safety infractions, are typically identified only once they have already caused disruption or cost.

Lack of a consolidated real-time operational view. Even where individual data points such as vehicle location or trip status may exist in isolated form, no single interface consolidates live GPS position, active trip progress, driver status, and vehicle health into one operational view. This absence of a centralized command view delays managers' ability to respond to emergencies, breakdowns, or route deviations as they occur, forcing coordination to rely on ad hoc communication rather than real-time situational awareness.

Absence of consolidated performance analytics. Fleet-wide metrics such as vehicle utilization rate, total distance traveled, fuel consumption trends, maintenance expenditure, downtime, driver rankings, and vehicle health scores are seldom calculated or presented in consolidated form. In their absence, operational and strategic decisions are frequently made on the basis of intuition or isolated anecdotes rather than aggregated evidence, limiting operators' ability to identify systemic inefficiencies or measure improvement over time.

Absence of predictive maintenance capability. Existing maintenance practices, even where scheduled, rely on fixed calendar intervals rather than condition- or usage-based prediction derived from mileage, engine hours, and historical repair patterns. Without predictive capability, operators cannot anticipate probable failures ahead of time, nor can they compare relative risk across vehicles through a standardized health score—resulting in missed opportunities for early, lower-cost intervention before a component fails in service.

Emerging gap in electric vehicle fleet readiness. As electric vehicles begin to enter commercial fleet operations, existing manual and semi-digital systems have no mechanism to monitor battery state of charge, estimate usable range, schedule charging, or track energy consumption and cost-per-kilowatt-hour. This represents a forward-looking gap that will become increasingly material as fleet composition diversifies.

Collectively, these deficiencies reflect a fleet management environment defined by fragmented data, reactive decision-making, and limited operational visibility. In the absence of a single integrated system capable of consolidating driver and vehicle records, trip and route data, maintenance and inventory management, fuel monitoring, accident and insurance administration, real-time operational oversight, and predictive analytics, transport operators remain exposed to preventable financial losses, safety risks, regulatory non-compliance, and diminished competitiveness. This gap between the operational demands of a growing logistics sector and the fragmented tools currently available to address them constitutes the central problem this project seeks to resolve.

3. General and Specific Objectives

3.1 General Objective

The general objective of this project is **to design and develop** a web-based Fleet Management System that consolidates driver and vehicle management, trip and route monitoring, preventive maintenance, fuel tracking, inventory control, accident and insurance administration, and predictive analytics into a single integrated platform for transport operators. The system is

intended to replace fragmented, manual fleet management practices with a centralized, data-driven solution that improves operational visibility, reduces maintenance and fuel-related costs, and enhances fleet safety, compliance, and reliability.

3.2 Specific Objectives

1. To design and implement a driver and vehicle management module that maintains profile records (specifications, licensing, and identification), assigns drivers to vehicles based on availability and qualification, tracks attendance and performance scores, and captures DVIR and accident report data.
2. To develop a trip and route management module that schedules and assigns trips, optimizes routes by distance and other parameters, tracks and reconciles mileage, monitors ETAs and delays, generates trip analytics, and provides simulated real-time GPS trip visualization.
3. To implement a fuel tracking module that records fuel purchases, reconciles consumption against mileage, computes cost-per-kilometer and efficiency trends, and detects abnormal usage indicative of theft or leakage.
4. To develop an alerts and notifications module that generates maintenance-due, document/license expiry, and simulated speeding, idling, and unauthorized-usage alerts, delivered through simulated email, SMS, and push notification channels.
5. To design a fleet analytics dashboard that calculates and visualizes utilization rate, distance traveled, downtime, and fuel and maintenance cost trends, and generates chart-based driver ranking and vehicle health score reports.
6. To develop a fleet command center featuring a simulated live GPS map of active vehicle positions, real-time trip and driver status, vehicle health indicators, and simulated traffic and weather overlays.
7. To develop an AI-based predictive maintenance feature that forecasts maintenance needs from mileage and engine-hour data and computes a standardized vehicle health score incorporating repair history and efficiency trends.

8. To implement an optional EV fleet management module that monitors battery state-of-charge and range, manages charging schedules, and tracks energy consumption, cost-per-kWh, and battery health.
9. To test and evaluate the developed system for functional accuracy, usability, and reliability in addressing the fleet management challenges identified in this proposal.

4. Scope & Limitations

4.1 Scope

The project is delimited to the design and development of a single, web-based Fleet Management , developed as an academic prototype for use by a single transport operator's fleet. To align development effort with the project's fixed academic timeframe, system functionality is organized into three tiers: a Minimum Viable Product (MVP) comprising the modules essential to the system's core tracking and maintenance-scheduling function; Optional/Phase-2 Enhancements that extend and enrich the MVP where time and resources permit; and functionality explicitly designated out of scope for the current project, retained only as a direction for potential future work.

- **Driver & Vehicle Management** — Vehicle and driver profile management, driver–vehicle assignment, attendance recording, performance scoring, DVIR submission, accident report logging, and document management with expiry alerts. As the foundational module supplying core entities referenced throughout the system, it is essential to the MVP.
- **Trip & Route Management** — Trip scheduling, route optimization, mileage tracking, ETA monitoring, delay reporting, trip analytics, and simulated real-time GPS tracking, representing the system's core fleet-tracking function.
- **Alerts & Notifications** — Maintenance-due and document/license expiry alerts, delivered via simulated email, SMS, and push channels. As scheduling-critical notifications tied directly to the Maintenance and Driver & Vehicle modules, these are

treated as part of the MVP; behavior-based alerts (speeding, idling, unauthorized usage), which depend on Fleet Command Center telemetry, are addressed under Phase-2.

- **Fuel Tracking**— Fuel purchase recording, consumption tracking, cost-per-kilometer calculation, efficiency trend analysis, and abnormal usage detection.
- **Fleet Analytics Dashboard** — Utilization rate, distance traveled, fuel and maintenance cost trends, downtime, driver rankings, and vehicle health scores, presented through charts and trend reports.
- **Fleet Command Center** — Consolidated simulated live GPS map, active trip status, driver status, vehicle health indicators, real-time alerts, and simulated traffic/weather conditions; also supplies the telemetry basis for behavior-based alerts.
- **EV Fleet Management** — Battery state-of-charge monitoring, range estimation, charging schedule management, and energy consumption/cost tracking, implemented only if time and resources remain.
- **AI Predictive Maintenance** — Mileage- and engine-hour-based prediction of maintenance needs and standardized vehicle health scoring. This component is identified as a valuable potential extension but falls outside the scope of the current project, given the additional data preparation, model development, and validation effort it would require beyond the available timeframe. Should sufficient historical maintenance data and development time become available in a future iteration, this capability could be incorporated as a natural extension of the Maintenance & Workshop Management module.

The system is further delimited to fleet and operations managers, dispatchers, and workshop personnel as its primary users, with driver-related data (attendance, performance, DVIR submissions) captured and managed within the system.

Out of Scope

Vehicle maintenance scheduling

Inventory management

...

4.2 Limitations

The following factors bound the depth to which certain features can be realized, or may hinder full attainment of the project's objectives within the available timeframe:

1. **Simulated GPS and real-time tracking.** Owing to the cost and logistics of installing physical GPS hardware in vehicles, live location, traffic, and weather data will be simulated using generated or sample data rather than sourced from actual devices or third-party live-tracking APIs.
2. **No integration with live external gateways.** Email, SMS, and push notifications will be demonstrated within the system's own interface rather than integrated with commercial SMS gateways or push notification services, which require paid subscriptions or provider agreements beyond the project's scope.
3. **No integration with vehicle onboard diagnostics (OBD-II).** Engine hours, fuel consumption, and fault data will be entered manually or simulated rather than retrieved from onboard vehicle sensors, as such integration requires specialized hardware unavailable to the project team.
4. **Limited training data for predictive maintenance.** The AI Predictive Maintenance component will be developed and validated using sample or synthetically generated data, as large-scale historical maintenance datasets from actual operators may not be accessible; prediction accuracy may consequently differ from what could be achieved with extensive real-world data.
5. **Single-organization scope.** The system is designed to serve one transport operator's fleet rather than function as a multi-tenant platform for multiple independent companies.
6. **No dedicated native mobile application.** The system will be delivered as a responsive web application; a native mobile application is outside the current scope.
7. **Optional module contingency.** EV Fleet Management will be implemented only if time and resources remain after the core modules are completed; its partial implementation or omission would not constitute a failure to meet the project's core objectives.

8. **Limited-scale testing.** Testing will be conducted using representative sample datasets rather than the data volume typical of an established large fleet, which may limit evaluation of performance under production-level load.
9. **Time and resource constraints.** The project is undertaken within a fixed academic timeframe by a small development team, which may constrain implementation depth for lower-priority features and limit the extent of field validation with an actual transport operator.
10. **Language and localization.** The system interface will be developed in English; localization into Amharic or other local languages is not included.

5. Methodology / Approach

5.1 Development Methodology

The project will follow an Agile, incremental development methodology, structured around iterative development cycles rather than a single linear build. This approach suits the system's modular architecture, in which largely independent but interrelated modules can be designed, developed, and tested in successive increments. Foundational modules—particularly Driver & Vehicle Management, which supplies core entities referenced throughout the system—will be prioritized early, with dependent modules (Trip & Route Management, Maintenance & Workshop Management, Fleet Analytics Dashboard, and others) built and integrated in subsequent iterations. This incremental structure allows continuous testing and refinement, early detection of design flaws, and demonstrable progress at each stage of the academic timeline.

5.2 Requirements Gathering and Analysis

Requirements will be informed by a review of literature and existing commercial fleet management platforms to identify standard functional expectations, supplemented by informal discussions with individuals familiar with transport and logistics operations to ground requirements in local operational realities. Functional and non-functional requirements will be documented and prioritized using a MoSCoW (Must have, Should have, Could have, Won't have) framework to distinguish core modules from optional extensions such as EV Fleet

Management. Use-case diagrams will define actor–system interactions for each module. *Tools:* Google Docs/Microsoft Word for requirement specifications; draw.io or Lucidchart for use-case diagrams.

5.3 System Design

System design will follow an **Object-Oriented Software Engineering (OOSE)** approach, in which Unified Modeling Language (UML) artifacts—including class, sequence, and activity diagrams—define system structure and behavior across modules. The relational database will be modeled using Entity-Relationship (ER) diagrams and normalized to at least third normal form (3NF) to minimize redundancy across interrelated entities such as vehicles, drivers, trips, maintenance records, and inventory items. System architecture will follow a layered (presentation–application–data) design exposed through a RESTful Application Programming Interface (API). User interface design will proceed through low- and high-fidelity wireframes and prototypes, validated before implementation begins.

Tools: draw.io/Lucidchart (UML diagrams, Microsoft Visio), MySQL Workbench (ER modeling), Figma (UI/UX prototyping).

5.4 System Implementation

Implementation will proceed module-by-module in alignment with the Agile iteration structure defined above. The system will be developed as a responsive web application using **React.js** with **Tailwind CSS** for the frontend, and **Node.js with Express.js** for the backend, communicating through a RESTful API. **MySQL** will serve as the relational database management system, selected for its suitability in modeling the structured, interrelated data typical of fleet operations. Simulated real-time features—including GPS position updates for the Fleet Command Center and live alert delivery—will be implemented using WebSocket-based communication (**Socket.io**), with simulated vehicle positions rendered on an interactive map using **Leaflet.js** with OpenStreetMap tiles. The Fleet Analytics Dashboard will use a charting library such as **Chart.js** for visualizing utilization, fuel, and cost trends. The AI Predictive Maintenance component(optional) will be developed separately in **Python**, using **pandas** for data preparation and **scikit-learn** for regression- or classification-based prediction, exposed to

the main application through a lightweight internal API if it's implemented. Source code will be managed under **Git**, hosted on **GitHub**, to support collaborative development and version control.

5.5 Environment Configuration

To support consistency across development, testing, and deployment environments, the system's components will be containerized using **Docker**. Each service — including the frontend, backend API, MySQL database, and the Python-based predictive maintenance service, where pursued — will be defined within its own isolated container, encapsulating its dependencies and runtime configuration independently of the host machine. This containerized approach reduces configuration drift between development machines, ensures the system behaves consistently across environments, and supports reproducible builds throughout the project lifecycle. Where multiple services must be coordinated — for example, running the frontend, backend, and database together during integration testing — **Docker Compose** will optionally be used to orchestrate multi-container startup, networking, and shared configuration through a single declarative definition.

5.6 Testing and Quality Assurance

Testing will be conducted at multiple levels. Unit testing will validate individual functions and components using **Jest** (JavaScript/React) and **PyTest** (Python-based predictive maintenance component) if the AI Predictive Maintenance is implemented . Integration testing will verify correct interaction between modules—for example, confirming that a completed work order correctly updates maintenance history and vehicle health scoring. API endpoints will be tested using **Postman** to validate request handling and error responses. System-level testing will assess the platform against the functional requirements defined during analysis, combining black-box test cases derived from use cases with white-box review of critical logic such as predictive scoring and alert triggering. Where feasible, informal user acceptance testing (UAT) will be conducted with individuals familiar with fleet operations to assess usability and workflow alignment.

5.7 Deployment and Documentation

Upon completion of testing, the system will be deployed to a cloud-based hosting environment suitable for academic demonstration, with the database hosted on a managed MySQL instance. Containerization using **Docker** will be considered to standardize the deployment environment. Technical documentation—including design artifacts, API documentation, and a user guide—will be compiled progressively throughout development rather than solely at the project's conclusion.

Table 1. Summary of Methods, Techniques, and Tools by Phase

Phase	Key Techniques	Key Tools
Requirements Gathering & Analysis	Literature review, practitioner consultation, MoSCoW prioritization, use-case modeling	Google Docs/Microsoft Word, draw.io, Lucidchart, Microsoft Project
System Design	UML modeling, ER modeling & normalization, RESTful API design, UI prototyping	draw.io, MySQL Workbench, Figma, Canva, Microsoft Visio
Implementation	Modular/incremental development, real-time WebSocket communication	React.js, Tailwind CSS, Node.js, Express.js, MySQL, Socket.io, Leaflet.js, Chart.js, Python (pandas, Scikit-learn), GitHub
Testing & QA	Unit, integration, black-box/white-box, and acceptance testing	Jest, PyTest, Postman

Deployment & Documentation	Cloud deployment, containerization, progressive documentation	Docker, cloud hosting platform, Markdown/Word
----------------------------	---	---

6. Significance & Beneficiaries

6.1 Significance of the Project

The significance of this project lies in demonstrating that the fragmented, reactive fleet management practices described in the Statement of the Problem can be replaced by a single, integrated, and proactive system. By consolidating driver and vehicle records, trip and maintenance data, fuel and inventory tracking, and predictive analytics into one platform, the system enables transport operators to shift from responding to breakdowns and cost overruns after they occur to anticipating and preventing them. The direct operational value of this shift includes reduced vehicle downtime through preventive and predictive maintenance, lower fuel and repair expenditure through consumption tracking and anomaly detection, improved regulatory compliance through automated document and license expiry alerts, and stronger driver safety and accountability through performance scoring and real-time alerting.

Beyond its immediate operational utility, the project holds significance as a contribution toward the digital transformation of Ethiopia's logistics sector discussed in the Introduction. It illustrates, at prototype scale, how locally grounded software—developed with awareness of the operational and resource constraints facing Ethiopian transport operators—can address needs that costly or poorly localized foreign platforms may not adequately serve. As an academic undertaking, the project further demonstrates the integration of several distinct software engineering and data science disciplines, including full-stack web development, relational database design, real-time systems, and predictive modeling, within a single coherent system, providing a reference point for future student work in fleet management or logistics informatics.

6.2 Beneficiaries of the Project

Beneficiary	Anticipated Benefit
Transport operators (fleet-owning organizations)	Reduced operational costs through fuel, maintenance, and inventory optimization; reduced vehicle downtime; improved regulatory compliance; stronger overall fleet reliability and competitiveness.
Fleet and operations managers / dispatchers	Centralized, real-time visibility into vehicle, driver, and trip status; data-driven decision-making through the analytics dashboard; faster response to disruptions via the Fleet Command Center.
Drivers	Clearer trip assignments and schedules, documented performance history supporting fair evaluation, and timely reminders for license and document renewal.
Clients of transport operators (indirect beneficiaries)	Improved delivery reliability and predictability resulting from better-managed fleets and reduced unplanned downtime.
Ethiopia's broader transport and logistics sector	A demonstrable, locally grounded reference model for fleet management digitalization, potentially reducing reliance on costly or poorly localized foreign platforms.

Academic community / future researchers	Project documentation and design artifacts serve as a reference for future work in fleet management systems, predictive maintenance, or logistics informatics.
The project team	Practical experience integrating full-stack development, database design, real-time systems, and predictive modeling—directly relevant to careers in logistics technology.

7. Task Breakdown & Feasibility

7.1 Task Breakdown

Phase 1: Requirements Gathering & Analysis

Task	Expected Outcome
Identify system stakeholders and user roles (fleet manager, dispatcher, mechanic, driver, administrator)	Defined actor list guiding subsequent use-case and access-control design
Review literature and existing commercial fleet management platforms	Comparative reference material informing feature scope and terminology
Conduct informal consultations with individuals familiar with transport/logistics operations	Contextual insight into operational pain points and priority features

Compile consultation and literature findings into a consolidated needs summary	Documented basis for requirement drafting
Draft functional requirements for each module (Driver & Vehicle, Trip & Route, Maintenance, Fuel, Inventory, Accident & Insurance, Alerts, Analytics, Command Center, Predictive Maintenance, EV)	Module-by-module functional requirement list
Draft non-functional requirements (performance, usability, security, scalability, availability)	Documented quality attributes guiding design decisions
Prioritize requirements using the MoSCoW framework	Requirements classified as Must/Should/Could/Won't have
Develop use-case diagrams for each module	Use-case diagram set defining actor–system interactions
Compile and internally validate the Software Requirements Specification (SRS)	Finalized SRS document serving as the reference baseline for design

Phase 2: System Design

Task	Expected Outcome
------	------------------

Design high-level system architecture (presentation–application–data layers, REST API)	Architecture diagram defining component boundaries
Design conceptual data model identifying core entities and relationships across modules	Conceptual entity map spanning vehicles, drivers, trips, maintenance, inventory, etc.
Develop Entity-Relationship (ER) diagram and normalize schema to at least 3NF	Normalized relational database schema
Develop UML class diagrams for core modules	Class-level structural design per module
Develop sequence diagrams for key workflows (trip lifecycle, work order lifecycle, alert triggering)	Behavioral design clarifying inter-component interaction
Develop activity diagrams for critical business processes (e.g., preventive maintenance triggering, DVIR-to-work-order flow)	Process-level design supporting implementation logic
Define REST API endpoints and request/response specifications per module	API specification document

Develop low-fidelity wireframes for key screens per module	Wireframe set covering primary user workflows
Develop high-fidelity UI prototypes and validate against requirements	Approved prototype ready for implementation
Conduct internal design review and finalize design documentation	Reviewed, finalized design package

Phase 3: Core Module Implementation

Driver & Vehicle Management

Task	Expected Outcome
Implement database schema for vehicle and driver profiles	Normalized tables storing specifications and credentials

Develop backend CRUD APIs for vehicle and driver records	Endpoints for creating, updating, retrieving, deactivating profiles
Implement driver–vehicle assignment logic based on availability and qualification	Assignment feature preventing conflicting or mismatched assignments
Implement attendance recording and performance scoring logic	Attendance log and computed per-driver performance score
Implement DVIR submission and review workflow	Digital DVIR form with backend storage and review status
Implement accident report submission feature	Structured report form linked to vehicle/driver records
Implement document upload with expiry tracking and automated alerts	Document repository triggering alerts ahead of expiry
Develop frontend interface for profiles, assignments, attendance, and DVIR/accident entry	Usable UI for fleet/HR staff

Trip & Route Management

Task	Expected Outcome
Implement trip scheduling and assignment logic (vehicle, driver, route, time)	Conflict-checked trip scheduling feature
Implement route optimization logic based on distance/route parameters	Suggested optimal route per trip
Implement mileage tracking and reconciliation against trip records	Per-trip and per-vehicle mileage totals
Implement ETA calculation and monitoring	Live ETA display updated against trip progress
Implement delay reporting with cause and duration capture	Structured delay records feeding analytics
Implement trip analytics aggregation (per driver, per route)	Summary views of trip performance trends
Implement simulated real-time GPS position feed	Generated position stream for map visualization

Develop frontend interface for scheduling, trip monitoring, and analytics	Usable UI for dispatchers
---	---------------------------

Fuel Tracking

Task	Expected Outcome
Implement fuel purchase recording feature	Logged fuel purchase records per vehicle
Implement fuel consumption tracking reconciled against mileage	Consumption figures tied to distance traveled
Implement cost-per-kilometer calculation	Per-vehicle cost-per-km metric
Implement fuel efficiency trend computation	Time-series efficiency data per vehicle
Implement abnormal usage detection and alerting logic	Flagged anomalies suggesting theft or leakage
Develop frontend interface for fuel logs and trend charts	Usable UI for fuel monitoring

Spare Parts & Inventory Management

Task	Expected Outcome
Implement stock level tracking database and logic	Real-time inventory quantities per part
Implement supplier and purchase order management	Supplier records and trackable purchase orders
Implement low-stock alert triggers	Automated reorder alerts
Implement parts-usage linkage to vehicles and work orders	Traceable parts consumption per repair
Implement warranty tracking	Warranty status visibility per part
Implement inventory cost analysis reporting	Cost summaries by part, supplier, and period
Develop frontend interface for inventory management	Usable UI for workshop/procurement staff

Alerts & Notifications

Task	Expected Outcome
Design centralized alert rule engine spanning all modules	Unified rule-evaluation mechanism
Implement maintenance-due and document/license expiry alerts	Automated alerts from Driver/Vehicle and Maintenance data
Implement speeding, idling, and unauthorized-usage alert logic (simulated telemetry)	Behavior-based alert triggers
Implement simulated email notification delivery	In-system email alert demonstration
Implement simulated SMS notification delivery	In-system SMS alert demonstration
Implement simulated push notification delivery	In-system push alert demonstration
Develop centralized notification management interface	Unified alert inbox/log for users

Task	Expected Outcome
Implement backend aggregation queries for utilization, distance, and downtime	Computed fleet-wide KPIs
Implement fuel and maintenance cost trend aggregation	Time-series cost data for visualization
Implement driver ranking computation	Ranked driver performance list
Implement vehicle health score aggregation	Consolidated per-vehicle health indicator
Develop frontend chart-based visualizations	Interactive dashboard charts
Implement dashboard filtering by date range, vehicle, and driver	Configurable, drill-down dashboard views

Fleet Command Center

Task	Expected Outcome
------	------------------

Implement simulated live GPS position feed via WebSocket	Streaming position updates
Develop interactive map interface displaying vehicle positions	Live map view of active vehicles
Implement active trip status panel	Real-time trip progress display
Implement driver status panel	Live driver availability/status view
Implement vehicle health indicator overlay	Health status visible alongside location
Implement real-time alert feed panel	Live alert stream within the command view
Implement simulated traffic/weather overlay	Contextual environmental data on the map

AI Predictive Maintenance

Task	Expected Outcome
------	------------------

Collect and prepare sample/synthetic training data (mileage, engine hours, repair history)	Structured dataset for model development
Perform data cleaning and feature engineering	Model-ready feature set
Select and train prediction model (regression/classification via scikit-learn)	Trained maintenance-prediction model
Validate model against test data and refine	Evaluated model with acceptable accuracy
Develop vehicle health scoring logic combining model output with rule-based factors	Standardized per-vehicle health score
Integrate prediction service with main application via internal API	Predictions accessible within the core system
Develop frontend interface displaying predictions and health scores	Usable UI presenting predictive insights

EV Fleet Management (Optional)

Task	Expected Outcome
Implement battery state-of-charge (SoC) monitoring (simulated)	Simulated SoC data per EV
Implement range estimation logic	Estimated remaining range per EV
Implement charging schedule management	Schedulable charging sessions
Implement energy consumption and cost-per-kWh tracking	Energy cost metrics per EV
Implement battery health tracking over time	Battery degradation trend view
Develop frontend interface for EV-specific metrics	Usable UI for EV monitoring, if implemented

Phase 4: Integration & Testing

Task	Expected Outcome
------	------------------

Conduct unit testing of backend logic per module	Verified correctness of individual functions
Conduct unit testing of frontend components	Verified UI component behavior
Conduct integration testing across interdependent modules (e.g., work order completion updating maintenance history and health score)	Verified cross-module data consistency
Conduct API testing using Postman	Validated request/response handling and error cases
Conduct system-level (black-box) testing against documented use cases	Test results mapped to functional requirements
Conduct white-box review of critical logic (predictive scoring, alert triggering, route optimization)	Verified internal logic correctness
Conduct informal user acceptance testing with individuals familiar with fleet operations	Usability feedback and identified refinements
Identify, prioritize, and resolve defects	Defect log with resolution status

Conduct regression testing after defect resolution	Confirmed stability following fixes
--	-------------------------------------

Phase 5: Deployment & Documentation

Task	Expected Outcome
Prepare production build and configure environment variables	Deployment-ready application build
Set up cloud hosting environment and deploy backend/frontend	Accessible hosted instance
Configure and migrate production database	Live database reflecting finalized schema
Conduct post-deployment smoke testing	Confirmed core functionality in production environment
Compile technical documentation (architecture, ER diagrams, API reference)	Complete technical documentation set

Compile user guide/manual	End-user reference document
Prepare final project report	Finalized written report
Prepare defense presentation and demonstration script	Presentation materials ready for defense

7.2 Feasibility Analysis

Technical Feasibility. The technologies selected for this project are mature, extensively documented, and widely adopted in both industry and academic settings, which substantially reduces technical risk. React.js and Tailwind CSS provide a component-based frontend architecture well suited to a system composed of eleven largely independent modules, allowing UI elements to be developed, tested, and reused across modules without duplicative effort. Node.js and Express.js support asynchronous, non-blocking request handling appropriate for a system that must serve concurrent API requests from multiple modules while also maintaining WebSocket connections for simulated real-time features. MySQL, as a relational database management system, is well matched to the system's data model, in which entities such as vehicles, drivers, trips, work orders, parts, and insurance policies are highly interrelated and benefit from enforced referential integrity, transactional consistency, and structured querying—for example, ensuring that a completed work order simultaneously updates maintenance history, decrements parts inventory, and recalculates a vehicle's health score without introducing data inconsistency.

Socket.io enables WebSocket-based simulated GPS position updates and real-time alert delivery without requiring physical GPS hardware or a paid location-tracking service, while Leaflet.js paired with OpenStreetMap tiles provides free, open-source map rendering, avoiding the

licensing costs associated with commercial mapping APIs at scale. The AI Predictive Maintenance component relies on standard, well-established regression and classification techniques available through scikit-learn, applied to structured tabular data (mileage, engine hours, repair history); such models do not require specialized computing infrastructure such as GPUs and can be trained and evaluated on standard development hardware.

The principal technical risk lies in maintaining data consistency across eleven interdependent modules during integration. This risk is mitigated through careful relational schema design with enforced foreign-key constraints, transactional database operations for multi-table updates, and a phased integration-testing strategy in which cross-module interactions are verified incrementally rather than only at final integration. Because all real-time and telemetry-dependent features (GPS, traffic, weather, speeding/idling detection) are simulated rather than sourced from physical hardware or third-party APIs, the most technically demanding and cost-intensive integration risks are avoided entirely.

Operational Feasibility. Operational feasibility is supported by the fact that system requirements were derived from a review of existing fleet management literature and platforms, combined with informal consultation with individuals familiar with transport and logistics operations, rather than assumed in isolation. This grounding increases the likelihood that the resulting workflows—trip scheduling, work order handling, document renewal tracking—reflect the actual sequence of tasks performed by fleet managers, dispatchers, and workshop staff. Use-case diagrams developed during the design phase further ensure that each module is built around clearly defined actor interactions rather than generic functionality.

As a browser-based responsive web application, the system requires no specialized hardware, proprietary devices, or software installation beyond a standard web browser and internet connectivity, making it realistically deployable within the resource constraints typical of small and medium-sized transport operators. The interface relies on familiar UI conventions—forms, tables, dashboards, and charts—already common to business software in use by administrative and workshop staff, lowering the anticipated learning curve.

Two operational risks are acknowledged. First, transitioning from manual or paper-based processes may encounter user resistance or habitual reliance on informal methods; this is

mitigated, within the project's scope, through intuitive interface design and clear demonstration of operational benefit, though full organizational change management lies beyond the project's boundaries. Second, system effectiveness depends on consistent data entry (e.g., logging fuel purchases, recording DVIRs); this is mitigated through streamlined, low-friction data entry forms designed to minimize end-user time burden. The modular architecture further allows incremental adoption—for example, beginning with Driver & Vehicle Management and Maintenance Scheduling before extending to Fuel and Inventory—easing the operational transition.

Economic Feasibility. The project's economic feasibility is strong, as all core technologies—React.js, Node.js, Express.js, MySQL, Python, scikit-learn, Socket.io, and Leaflet.js—are open-source and released under permissive licenses that impose no direct financial cost. Supporting design and modeling tools, including Figma, draw.io, Lucidchart, and MySQL Workbench, offer free tiers sufficient for the scope of a single-team academic project, and GitHub is free for repositories of this scale. Deployment for demonstration purposes can be accomplished using free or low-cost cloud tiers, avoiding the infrastructure costs of production-scale commercial hosting.

Because GPS tracking, vehicle telemetry, and notification delivery are simulated rather than integrated with commercial providers, the project avoids recurring third-party service costs—such as per-message SMS gateway fees or paid mapping API usage—that would otherwise represent the most significant ongoing expense in a production fleet management system. The absence of physical GPS or OBD-II hardware integration eliminates hardware procurement costs entirely. The principal cost is therefore the development team's time and effort, treated as an inherent academic requirement rather than an external financial cost—yielding a highly favorable ratio of anticipated value to required investment.

Schedule Feasibility. The project's phased, incremental methodology is structured to fit within a fixed academic timeframe while accommodating the scope implied by eleven functional modules. Requirements gathering and system design precede implementation to minimize costly rework, while MoSCoW prioritization ensures that foundational, "Must have" functionality—Driver & Vehicle Management, Trip & Route Management, core Maintenance scheduling, and basic Alerts—is completed first, followed by "Should/Could have" functionality

such as the Analytics Dashboard, Fleet Command Center, and Predictive Maintenance, with EV Fleet Management scheduled last.

This structure ensures that, even under time pressure, a coherent and functionally complete subset of the system can be demonstrated at the project's conclusion. Each phase is intended to conclude with a checkpoint suitable for advisor review, allowing schedule deviations to be identified early. The primary schedule risk stems from the breadth of scope relative to a single academic term; this is mitigated through the prioritization framework, time-boxing of each phase, and the explicit treatment of EV Fleet Management as optional and lowest-priority.

Resource Feasibility. Resource requirements are modest and realistically attainable.

Development requires only standard personal computing hardware capable of running a modern code editor, local development server, and MySQL instance—specifications met by typical student-owned laptops—together with consistent internet access. Because AI Predictive Maintenance relies on classical machine learning algorithms applied to structured, moderately sized datasets rather than deep learning on large-scale data, model training does not require GPUs or cloud computing clusters.

Software resource requirements are similarly modest, as all core frameworks and tools are freely available without procurement delays. Training and testing data can be generated synthetically or adapted from publicly available vehicle and fleet datasets, removing dependence on obtaining proprietary operational data from an actual transport operator. Human resource requirements are limited to the project team's own technical effort, supplemented by periodic academic advisor guidance and informal practitioner input. No specialized laboratory space, physical vehicles, or third-party personnel are required.

Legal and Ethical Feasibility. Although the system manages data types that would be sensitive in a live deployment—driver identification and licensing details, performance scores, and accident records—the project's academic, prototype-stage scope substantially limits associated legal and ethical exposure. Development and testing will rely exclusively on synthetic or sample data rather than real personal information, avoiding data protection concerns that would arise from processing genuine personal records. Basic security practices—authentication, role-based access control, and encrypted credential storage—will nonetheless be incorporated to reflect

responsible data-handling practices and to keep the architecture compatible with future compliance efforts should the system be extended toward real-world deployment. As the system is not intended for commercial release within this project's scope, formal regulatory registration is not required at this stage.

Table 2. Key Risks and Mitigations by Feasibility Dimension

Dimension	Key Risk	Mitigation
Technical	Data inconsistency across interdependent modules during integration	Enforced foreign-key constraints, transactional updates, phased integration testing
Operational	Resistance to shifting from manual processes; inconsistent data entry	Intuitive, low-friction UI; incremental, module-by-module adoption
Economic	Reliance on any paid service inflating cost	Exclusive use of open-source tools and free-tier hosting; simulated third-party integrations
Schedule	Scope breadth (11 modules) relative to academic term length	MoSCoW prioritization, phase checkpoints, EV module deprioritized
Resource	Limited access to real operator data for testing	Use of synthetic/sample datasets

Legal & Ethical	Handling of sensitive driver/vehicle data	Synthetic data only; role-based access control built into design
-----------------	---	--

8. Project Schedule / Timeline

The project is scheduled across five sequential phases, aligned with the task breakdown in Section 7, spanning the fixed window from **July 30 to November 30** — a total of approximately 17 weeks (~4 months). Scheduling follows Gantt-chart logic, with each phase's duration sized to the task volume identified in Section 7.1, and sequenced according to module dependencies consistent with PERT-style critical-path reasoning: modules that supply core data consumed by other modules are scheduled first, while modules that aggregate or model data from earlier modules are scheduled last.

8.1 PERT Three-Point Duration Estimates

To validate the duration assigned to each phase, optimistic (O), most likely (M), and pessimistic (P) estimates were used to calculate expected duration via $TE = (O + 4M + P) / 6$:

Phase	Optimistic	Most Likely	Pessimistic	Expected (TE)
Requirements Gathering & Analysis	10 days	14 days	18 days	14 days
System Design (OOSE)	10 days	14 days	20 days	14 days

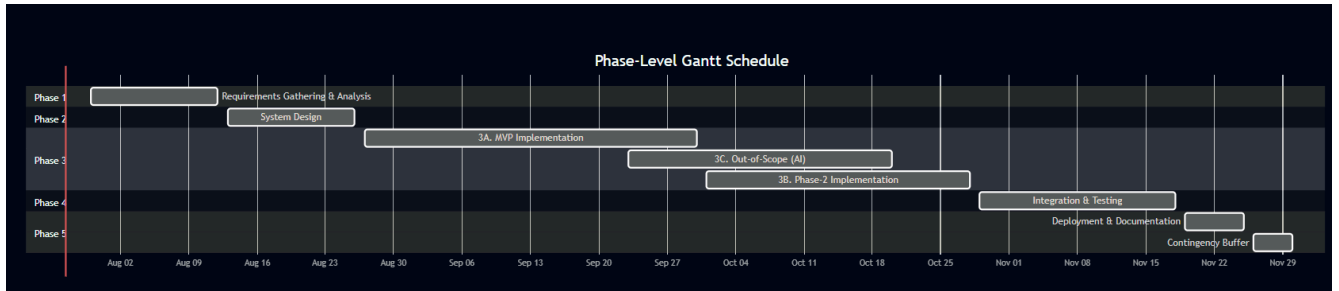
Environment Configuration	2 days	3 days	5 days	3 days
MVP Module Implementation	28 days	35 days	45 days	~36 days
Phase-2 Module Implementation	14 days	19 days	25 days	~19 days
AI Predictive Maintenance	21 days	28 days (4 wk)	35 days	28 days
Integration & Testing	15 days	21 days	28 days	~21 days

The most-likely (Gantt) durations used below track these expected values closely, giving reasonable confidence in the plan.

8.2 Phase-Level Gantt Schedule

Phase	Duration	Start	End	Deliverable
1. Requirements Gathering & Analysis	2 weeks	Aug 25	Sep 7	Finalized SRS, use-case diagrams

2. System Design	2 weeks	Sep 8	Sep 21	UML/ER diagrams, API spec, UI prototypes
3A. MVP Module Implementation	5 weeks	Sep 22	Oct 26	Functional MVP
3C. AI Predictive Maintenance <i>(parallel track)</i>	4 weeks	Oct 19	Nov 15	Prototype prediction model
3B. Phase-2 Module Implementation <i>(contingent)</i>	3 weeks	Oct 27	Nov 14	Functional Phase-2 enhancements, scope-dependent
4. Integration & Testing	3 weeks	Nov 15	Dec 5	Integrated, tested system build
5. Deployment & Documentation	1 week	Dec 6	Dec 12	Deployed system, final report, defense materials
Contingency Buffer	~5 days	Dec 13	Dec 17	Schedule slack, final revisions



8.3 MVP Implementation Schedule (Phase 3A Detail)

Task / Module	Duration	Start	End
Environment Configuration	3 days	Sep 22	Sep 24
Driver & Vehicle Management	9 days	Sep 22	Sep 30
Trip & Route Management	9 days	Oct 1	Oct 9
Maintenance & Workshop Management	9 days	Oct 10	Oct 18
Alerts & Notifications (<i>MVP subset</i>)	8 days	Oct 19	Oct 26

8.4 Phase-2 Implementation Schedule (Phase 3B Detail — Contingent)

Task / Module	Duration	Start	End
Fuel Management	5 days	Oct 27	Oct 31
Fleet Analytics Dashboard	5 days	Nov 1	Nov 5
Fleet Command Center (<i>incl. behavior-based alerts</i>)	5 days	Nov 6	Nov 10
EV Fleet Management (<i>lowest priority</i>)	4 days	Nov 11	Nov 14

8.5 Out-of-Scope Implementation Schedule (Phase 3C Detail)

Task	Duration	Start	End
Prepare sample/synthetic training data	6 days	Oct 19	Oct 24
Feature engineering and model selection	6 days	Oct 25	Oct 30
Train and validate prediction model (scikit-learn)	9 days	Oct 31	Nov 8

Develop vehicle health scoring logic and internal API integration	7 days	Nov 9	Nov 15
---	--------	-------	--------

This track begins only once the Maintenance & Workshop Management schema (Sep 22) is available, since predictive scoring depends on its repair-history data. It requires a team member — or dedicated portion of team time — able to work on the Python/scikit-learn component independently of the main React/Node/MySQL build; if no such capacity exists, this track should be dropped without affecting any other phase's dates, since nothing downstream depends on it finishing.

8.6 Critical Path & Float Analysis

The critical path — the minimum sequence required to deliver a fully functional MVP — runs Requirements → Design → Environment Configuration/MVP Implementation → Integration & Testing → Deployment & Documentation, totaling 91 days (13 weeks). Environment Configuration (3 days) is scheduled in parallel with the start of MVP Implementation rather than before it, so its duration is absorbed within the longer implementation activity and does not extend the critical path.

This leaves approximately 23 days of float relative to the December 17 maximum end date — schedule slack not required for MVP delivery. This float is allocated to Phase-2 Implementation (19 days) plus the final contingency buffer (4–5 days). Because Phase-2 modules sit outside the critical path, delays there do not threaten the MVP deadline: should MVP Implementation run over its estimate, the shortfall is absorbed first from the Phase-2 allocation — beginning with EV Fleet Management, then Fleet Command Center, in reverse priority order — before drawing on the final buffer.

The Out-of-Scope Implementation track (Oct 19 – Nov 15) runs entirely within the window already occupied by the tail of MVP and the bulk of Phase-2. Though it finishes one day after

Phase-2 itself ends (Nov 14), it does not extend the critical path or draw on the float held by the final contingency buffer.

Testing is not purely deferred to Phase 4: consistent with the Agile methodology, unit and integration tests are conducted incrementally as each module is completed; Phase 4 represents a dedicated system-level and acceptance-testing pass across the fully integrated build.

8.7 Key Milestones

Milestones	Target Date
M1 — SRS finalized	Sep 7
M2 — Design approved (UML/ER/API/UI prototypes)	Sep 21
M3 — Development environment configured	Sep 24
M4 — MVP feature-complete	Oct 26
M5 — AI Predictive Maintenance prototype complete (<i>parallel track, if pursued</i>)	Nov 14
M6 — Phase-2 enhancements complete	Nov 15
M7 — System testing complete	Dec 5

M8 — Deployment and documentation complete	Dec 12
M9 — Final submission	By Dec 17

9. References

1. Mohammed, A. E., "Senior Project Documentation Guideline (Version 1.4.3)," HiLCOE School of Computer Science & Technology, Addis Ababa, 2024, pp. 1-20.
2. Bruegge, B., & Dutoit, A. H. (2009). *Object-oriented software engineering: Using UML, patterns, and Java* (3rd ed.). Prentice Hall.
3. Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.
4. Pressman, R. S., & Maxim, B. R. (2015). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill Education.
5. Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for agile software development*. <https://agilemanifesto.org/>
6. Automotive Fleet. (n.d.). The history of computers in fleet management. Automotive Fleet.
7. Boylan, J., Meyer, D., & Chen, W. S. (2024). A systematic review of the use of in-vehicle telematics in monitoring driving behaviours. *Accident Analysis & Prevention*, 199, 107519. <https://doi.org/10.1016/j.aap.2024.107519>
8. Cheung, B. K.-S., Choy, K. L., Li, C.-L., Shi, W., & Tang, J. (2008). Dynamic routing model and solution methods for fleet management with mobile technologies. *International Journal of Production Economics*, 113(2), 694–705. <https://doi.org/10.1016/j.ijpe.2007.10.018>
9. Crainic, T. G., & Laporte, G. (1997). Planning models for freight transportation. *European Journal of Operational Research*, 97(3), 409–438. [https://doi.org/10.1016/S0377-2217\(96\)00298-6](https://doi.org/10.1016/S0377-2217(96)00298-6)

10. Crainic, T. G., & Laporte, G. (Eds.). (1998). Fleet management and logistics. Springer.
<https://link.springer.com/book/10.1007/978-1-4615-5755-5>
11. Fayyad, J., Jaradat, M. A., Gruyer, D., & Najjaran, H. (2020). Deep learning sensor fusion for autonomous vehicle perception and localization: A review. *Sensors*, 20(15), 4220.
<https://doi.org/10.3390/s20154220>
12. Goel, A. (2008). Fleet telematics: Real-time management and planning of commercial vehicle operations. Springer. <https://doi.org/10.1007/978-0-387-75105-4>
13. Hu, J., Morais, H., Sousa, T., & Lind, M. (2016). Electric vehicle fleet management in smart grids: A review of services, optimization and control aspects. *Renewable and Sustainable Energy Reviews*, 56, 1207–1226. <https://doi.org/10.1016/j.rser.2015.12.014>
14. Killeen, P., Ding, B., Kiringa, I., & Yeap, T. (2019). IoT-based predictive maintenance for fleet management. *Procedia Computer Science*, 151, 607–613.
<https://doi.org/10.1016/j.procs.2019.04.184>
15. Nisyrios, E., & Gkiotsalitis, K. (2025). Optimization-based approaches for traffic and fleet management of connected and autonomous vehicles: A systematic literature review. *International Journal of Intelligent Transportation Systems Research*, 23, 1668–1690.
<https://doi.org/10.1007/s13177-025-00536-2>
16. Pillac, V., Gendreau, M., Guéret, C., & Medaglia, A. L. (2013). A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 225(1), 1–11.
<https://doi.org/10.1016/j.ejor.2012.08.015>
17. SAE International. (2024). SAE J1939/73: Application layer—Diagnostics. SAE International.
18. Toledo, T., Musicant, O., & Lotan, T. (2016). Can feedback from in-vehicle data recorders improve driver behavior and reduce fuel consumption? *Transportation Research Part A: Policy and Practice*, 94, 194–204. <https://doi.org/10.1016/j.tra.2016.09.001>
19. Walnum, H. J., & Simonsen, M. (2015). Does driving behavior matter? An analysis of fuel consumption data from heavy-duty trucks. *Transportation Research Part D: Transport and Environment*, 36, 107–120. <https://doi.org/10.1016/j.trd.2015.02.016>
20. Yeong, D. J., Velasco-Hernandez, G., Barry, J., & Walsh, J. (2021). Sensor and sensor fusion technology in autonomous vehicles: A review. *Sensors*, 21(6), 2140.
<https://doi.org/10.3390/s21062140>

21. International Organization for Standardization. (2020). ISO/TS 15143-3:2020: Earth-moving machinery and mobile road construction machinery—Worksite data exchange—Part 3: Telematics data. ISO. <https://www.iso.org/standard/76394.html>
22. Webfleet. (2026). Fleet management software.
https://www.webfleet.com/en_us/webfleet/products/webfleet/
23. Verizon Connect. (2026). GPS fleet tracking software and fleet management solutions.
<https://www.verizonconnect.com/solutions/gps-fleet-tracking-software/>
24. Motive. (2026). Fleet telematics and fleet management platform.
<https://gomotive.com/products/features/fleet-telematics/>
25. Samsara. (2026). Fleet telematics and fleet management.
<https://www.samsara.com/products/telematics>
26. Geotab. (2026). Platform overview: AI-driven platform for connected fleet management.
<https://www.geotab.com/products/platform-overview/>